```
sub_method(123, "cat")
```

produces:

```
File            Line  Source Line
---------------+----+------------
main.rb         :  5 - dump_call_stack
sub.rb          :  2 - main_method(v1 * 3, v2 * 6)
main.rb         :  8 - sub_method(123, "cat")
```

The SCRIPT_LINES__ constant is closely related to this technique. If a program initializes a constant called SCRIPT_LINES__ with a hash, that hash will receive a new entry for every file subsequently loaded into the interpreter using require or load. The entry's key is the name of the file, and the value is the source of the file as an array of strings.

Behind the Curtain: The Ruby VM

If you'd like to know what Ruby is doing with all that code you're writing, you can ask the Ruby interpreter to show you the intermediate code that it is executing.

You can ask it to compile the Ruby code in a string or in a file and then disassemble it and even run it. You might wonder if it can dump the opcodes out and later reload them. The answer is no—the interpreter has the code to do this, but it is disabled because there is not yet an intermediate code verifier for YARV.

Here's a trivial example of disassembly:

```
code = RubyVM::InstructionSequence.compile('a = 1; puts 1 + a')
puts code.disassemble
```

produces:

```
== disasm: #<ISeq:<compiled>@<compiled>:1 (1,0)-(1,17)> (catch: false)
local table (size: 1, argc: 0 [opts: 0, rest: -1, post: 0, block: -1, kw: -1@-1,
kwrest: -1])
[ 1] a@0
0000 putobject_INT2FIX_1_                                             (   1)[Li]
0001 setlocal_WC_0                          a@0
0003 putself
0004 putobject_INT2FIX_1_
0005 getlocal_WC_0                          a@0
0007 opt_plus                               <calldata!mid:+, argc:1,
ARGS_SIMPLE>[CcCr]
0009 opt_send_without_block                 <calldata!mid:puts, argc:1,
FCALL|ARGS_SIMPLE>
0011 leave
```

Maybe you want to know how Ruby handles #{...} substitutions in strings. Ask the VM:

```
code = RubyVM::InstructionSequence.compile('a = 1; puts "a = #{a}."')
puts code.disassemble
```

produces:

```
== disasm: #<ISeq:<compiled>@<compiled>:1 (1,0)-(1,23)> (catch: false)
local table (size: 1, argc: 0 [opts: 0, rest: -1, post: 0, block: -1, kw: -1@-1,
kwrest: -1])
[ 1] a@0
0000 putobject_INT2FIX_1_                                             (   1)[Li]
```